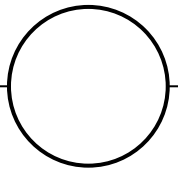


Sistemas Numéricos

Forense Computacional

(IMDo718)

Prof. Dr. Ramon dos Reis Fontes
ramon.fontes@imd.ufrn.br



Overview

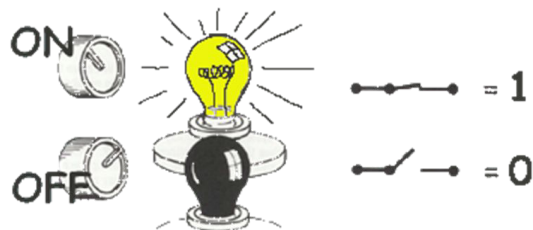
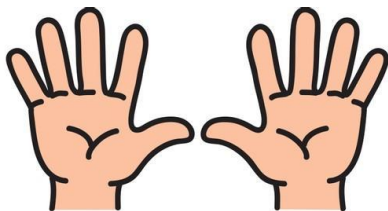
- Sistemas numéricos e conversão
- Data, formato de data, conversão de data
- Código Padrão Americano para Intercâmbio de Informações (código ASCII)
- Dados armazenados na memória

Sobre o envio do laboratório

- Considere a folha de respostas para o Lab1 disponível no gdrive

Sistemas Numéricos

10
ten



System	Base	Symbols	Used by humans?	Used by CPU?	Used by Forensic Professionals?
Decimal	10	0, 1, ... 9	Yes	No	Yes
Binary	2	0, 1	No	Yes	Yes
Hexa-decimal	16	0, 1, ... 9, A, B, ... F	No	No	Yes

Conversão entre sistemas numéricos

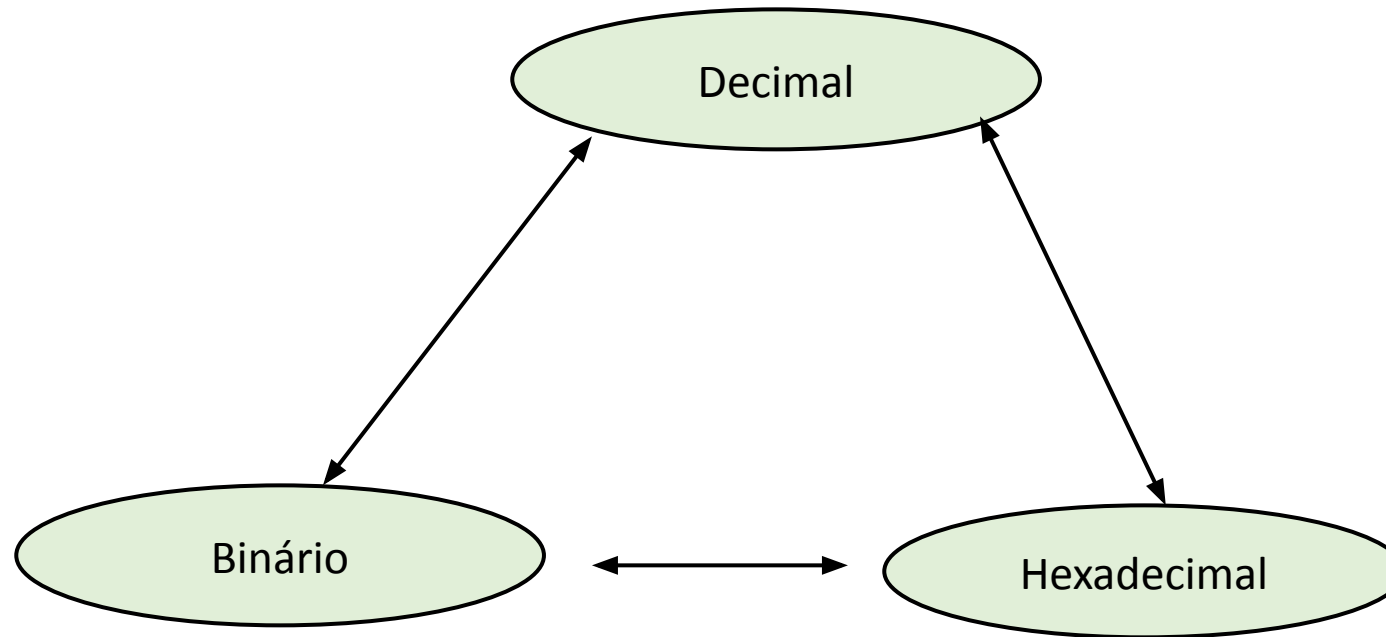


Tabela de conversão

Decimal	Binário	Hexa-decimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7

Decimal	Binário	Hexa-decimal
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Adição Binária

A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	10

carregar
bit

$$\begin{array}{r} ^1 ^1 \\ 10101 \\ + 11001 \\ \hline 101110 \end{array}$$

$$\begin{array}{r} 21 \\ + 25 \\ \hline 46 \end{array}$$

Decimal para Decimal

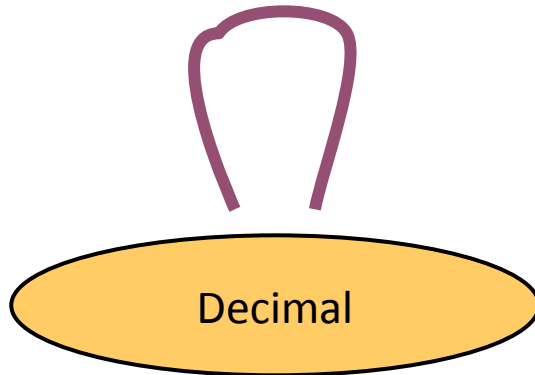


Diagram illustrating the conversion of the decimal number 126 to its positional weights (Base 10) and the resulting values (Peso).

The diagram shows the decomposition of 126 into its positional weights:

- 126₁₀ is decomposed into 6, 2, and 1.
- Each digit is multiplied by its corresponding power of 10 (Peso):
 - 6 × 10⁰ = 6
 - 2 × 10¹ = 20
 - 1 × 10² = 100

The final result is the sum of these products: 6 + 20 + 100 = 126.

Annotations:

- espaço reservado** (reserved space) points to the space between the digit and the multiplication symbol.
- Peso** (Weight) points to the power of 10.
- Base** points to the base 10.

Binário para Decimal

- Multiply each bit by 2^n , where n is the “weight” of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

Diagram illustrating the conversion of the binary number 101010_2 to decimal using positional weights.

The binary number is 101010_2 . The weights are powers of 2, starting from 2^0 on the right.

Calculation:

Bit	Weight	Product
0	2^0	0
1	2^1	2
0	2^2	0
1	2^3	8
0	2^4	0
1	2^5	32

The sum of the products is 42_{10} .

Labels: **espaço reservado** (reserved space), **peso** (weight), **Base**.

```
(kali@kali)-[~]  
$ echo "$((2#101010))"  
42
```

$\$(())$: Shell Arithmetic
 $2\#$ base 2

Binário para Decimal (código Python)

Check Python version

```
(student@kalit01)-[~]  
$ python --version  
Python 2.7.18  
  
(student@kalit01)-[~]  
$ python3 --version  
Python 3.9.13
```

I use Python version 3.9

```
(student@kalit01)-[~]  
$ python3  
Python 3.9.13 (main, Jun 8 2022, 09:45:57)  
[GCC 11.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> int("101010",2)  
42  
>>>
```

hex to int
int("800", 16)

Hexadecimal para Decimal

- Multiply each bit by 16^n , where n is the “weight” of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

$$\begin{array}{rcll} \text{AB2}_{16} & \Rightarrow & (?)_{10} & \\ 2 \times 16^0 & = & 2 \times 1 & = 2 \\ \text{B} \times 16^1 & = & 11 \times 16 & = 176 \\ \text{A} \times 16^2 & = & 10 \times 256 & = 2560 \\ & & & \hline & & & 2738_{10} \end{array}$$

```
(kali㉿kali)-[~]  
$ echo "$((16#AB2))"  
2738
```

```
(kali㉿kali)-[~]  
$ echo $((0xAB2))  
2738  
  
(kali㉿kali)-[~]  
$ echo $((5*5))  
25
```

$\$()$: Shell Arithmetic

Decimal para Binário

- Divide by **2**, keep track of the remainder
- First remainder is bit 0 (LSB, least-significant bit)
- Second remainder is bit 1
- Etc.

```
(kali⊗kali)-[~]  
$ sudo apt install bc
```

basic
calculator

$$126_{10} = (?)_2$$

2	126	
2	63	0
2	31	1
2	15	1
2	7	1
2	3	1
2	1	1
stop	→ 0	1



$$126_{10} = 1111110_2$$

```
(kali⊗kali)-[~]  
$ echo "obase=2;126" | bc  
1111110
```

Decimal to Hexadecimal

- Divide by **16**
- Keep track of the remainder

$$2738_{10} = (?)_{16}$$

16		2738	
16		171	2
16		10	11 = B
		0	10 = A

$$2738_{10} = AB2_{16}$$

```
(kali㉿kali)-[~]  
$ echo "obase=16;2738" | bc  
AB2
```

```
(kali㉿kali)-[~]  
$ printf "%x" "2738"  
ab2
```

controls the output as in C printf

%d - Print the argument as a signed decimal integer.

%x - Print the argument as an unsigned hexadecimal integer.

%s - Print the argument as a string.

Decimal to Binário and Hex (Código Python)

Decimal to Binary

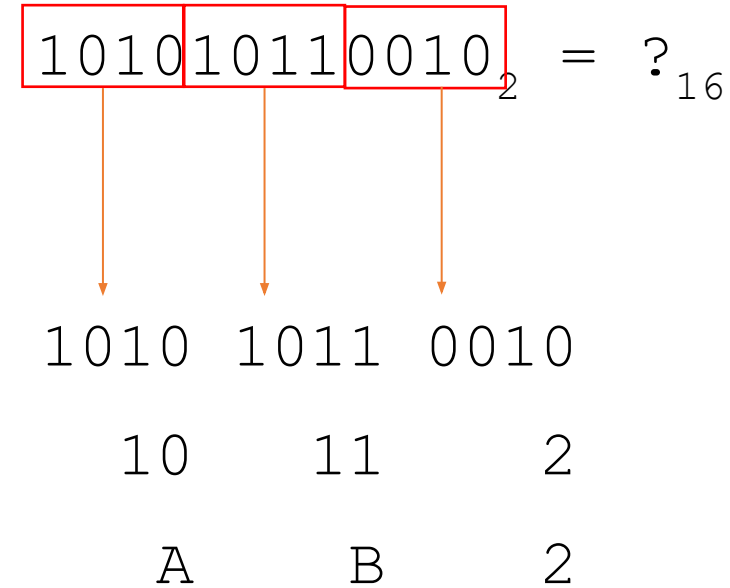
```
>>> bin(126)
'0b1111110'
>>> 
```

Decimal to Hex

```
>>> hex(2739)
'0xab3'
>>> 
```

Binário para Hexadecimal

- Group bits in **fours**, starting on **right**
- Convert to hexadecimal digits



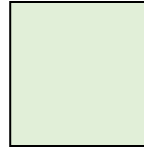
$$101010110010_2 = AB2_{16}$$

Código Padrão Americano para Intercâmbio de Informações

Código ASCII

Bit, Byte

- A 'bit' (from **B**inary + dig**I**T)
 - is the smallest unit of memory,
 - the unit of measurement of data information.
- Byte 8-bit collections
 - A single bit holds so little information.
 - A byte holds one ASCII



ASCII - American Standard Code for Information Interchange

- ASCII codes represent text in computing devices.

ASCII control characters			ASCII printable characters			Extended ASCII characters				
00	NULL	(Null character)	32	space	64	@	96	`	128	Ç
01	SOH	(Start of Header)	33	!	65	A	97	a	129	ü
02	STX	(Start of Text)	34	"	66	B	98	b	130	é
03	ETX	(End of Text)	35	#	67	C	99	c	131	â
04	EOT	(End of Trans.)	36	\$	68	D	100	d	132	ä
05	ENQ	(Enquiry)	37	%	69	E	101	e	133	à
06	ACK	(Acknowledgement)	38	&	70	F	102	f	134	á
07	BEL	(Bell)	39	'	71	G	103	g	135	ç
08	BS	(Backspace)	40	(	72	H	104	h	136	ê
09	HT	(Horizontal Tab)	41	)	73	I	105	i	137	ë
10	LF	(Line feed)	42	*	74	J	106	j	138	è
11	VT	(Vertical Tab)	43	+	75	K	107	k	139	ï
12	FF	(Form feed)	44	,	76	L	108	l	140	î
13	CR	(Carriage return)	45	-	77	M	109	m	141	ï
14	SO	(Shift Out)	46	.	78	N	110	n	142	Ä
15	SI	(Shift In)	47	/	79	O	111	o	143	Å
16	DLE	(Data link escape)	48	0	80	P	112	p	144	É
17	DC1	(Device control 1)	49	1	81	Q	113	q	145	æ
18	DC2	(Device control 2)	50	2	82	R	114	r	146	Æ
19	DC3	(Device control 3)	51	3	83	S	115	s	147	ó
20	DC4	(Device control 4)	52	4	84	T	116	t	148	ö
21	NAK	(Negative acknowl.)	53	5	85	U	117	u	149	ò
22	SYN	(Synchronous idle)	54	6	86	V	118	v	150	û
23	ETB	(End of trans. block)	55	7	87	W	119	w	151	ù
24	CAN	(Cancel)	56	8	88	X	120	x	152	ÿ
25	EM	(End of medium)	57	9	89	Y	121	y	153	Ö
26	SUB	(Substitute)	58	:	90	Z	122	z	154	Û
27	ESC	(Escape)	59	;	91	[	123	{	155	ø
28	FS	(File separator)	60	<	92	\	124		156	£
29	GS	(Group separator)	61	=	93	]	125	}	157	Ø
30	RS	(Record separator)	62	>	94	^	126	~	158	×
31	US	(Unit separator)	63	?	95	_			159	f
127	DEL	(Delete)							160	á
									161	í
									162	ó
									163	ú
									164	ñ
									165	Ñ
									166	ª
									167	º
									168	¿
									169	®
									170	¬
									171	½
									172	¼
									173	¡
									174	«
									175	»
									176	░
									177	▒
									178	▓
									179	▒
									180	┐
									181	À
									182	Á
									183	Â
									184	Ã
									185	Ä
									186	Å
									187	┐
									188	┐
									189	¢
									190	¥
									191	¬
									192	Ł
									193	ł
									194	┐
									195	┐
									196	—
									197	+
									198	ä
									199	Å
									200	Ł
									201	┐
									202	┐
									203	┐
									204	┐
									205	=
									206	┐
									207	▒
									208	ð
									209	Ð
									210	Ê
									211	Ë
									212	È
									213	Ì
									214	Í
									215	Î
									216	Ï
									217	┐
									218	┐
									219	┐
									220	┐
									221	┐
									222	┐
									223	┐
									224	Ó
									225	Ô
									226	Õ
									227	Ö
									228	ö
									229	Ø
									230	μ
									231	þ
									232	Þ
									233	Ú
									234	Û
									235	Ü
									236	Ý
									237	Ý
									238	—
									239	·
									240	≡
									241	±
									242	≡
									243	¼
									244	¶
									245	§
									246	÷
									247	·
									248	·
									249	·
									250	·
									251	·
									252	·
									253	·
									254	■
									255	nbsp

NAME

ascii - ASCII character set encoded in octal, decimal, and hexadecimal

DESCRIPTION

ASCII is the American Standard Code for Information Interchange. It is a 7-bit code. Many 8-bit codes (e.g., ISO 8859-1) contain ASCII as their lower half. The international counterpart of ASCII is known as ISO 646-IRV.

The following table contains the 128 ASCII characters.

C program '\X' escapes are noted.

Oct	Dec	Hex	Char	Oct	Dec	Hex	Char
000	0	00	NUL '\0' (null character)	100	64	40	@
001	1	01	SOH (start of heading)	101	65	41	A
002	2	02	STX (start of text)	102	66	42	B
003	3	03	ETX (end of text)	103	67	43	C
004	4	04	EOT (end of transmission)	104	68	44	D
005	5	05	ENQ (enquiry)	105	69	45	E
006	6	06	ACK (acknowledge)	106	70	46	F
007	7	07	BEL '\a' (bell)	107	71	47	G
010	8	08	BS '\b' (backspace)	110	72	48	H
011	9	09	HT '\t' (horizontal tab)	111	73	49	I
012	10	0A	LF '\n' (new line)	112	74	4A	J
013	11	0B	VT '\v' (vertical tab)	113	75	4B	K
014	12	0C	FF '\f' (form feed)	114	76	4C	L
015	13	0D	CR '\r' (carriage ret)	115	77	4D	M
016	14	0E	SO (shift out)	116	78	4E	N
017	15	0F	SI (shift in)	117	79	4F	O
020	16	10	DLE (data link escape)	120	80	50	P
021	17	11	DC1 (device control 1)	121	81	51	Q
022	18	12	DC2 (device control 2)	122	82	52	R
023	19	13	DC3 (device control 3)	123	83	53	S
024	20	14	DC4 (device control 4)	124	84	54	T
025	21	15	NAK (negative ack.)	125	85	55	U
026	22	16	SYN (synchronous idle)	126	86	56	V
027	23	17	ETB (end of trans. blk)	127	87	57	W
030	24	18	CAN (cancel)	130	88	58	X
031	25	19	EM (end of medium)	131	89	59	Y
032	26	1A	SUB (substitute)	132	90	5A	Z
033	27	1B	ESC (escape)	133	91	5B	[
034	28	1C	FS (file separator)	134	92	5C	\ '\\'
035	29	1D	GS (group separator)	135	93	5D	]
036	30	1E	RS (record separator)	136	94	5E	^
037	31	1F	US (unit separator)	137	95	5F	_



Conversão de número e ASCII

Hex to ASCII string (1)

```
(kali㉿kali)-[~]  
$ printf "\x48"  
H  
  
(kali㉿kali)-[~]  
$ printf "\x48\x65\x6c\x6c\x6f"  
Hello
```

Hex to string

Decimal to ASCII string (2)

```
(kali㉿kali)-[~]  
$ printf "\x$(printf "%x" "72")"  
H
```

ASCII to Hex (3)

```
(kali㉿kali)-[~]  
$ echo H | xxd  
00000000: 480a  
  
(kali㉿kali)-[~]  
$ echo Hello | xxd  
00000000: 4865 6c6c 6f0a
```

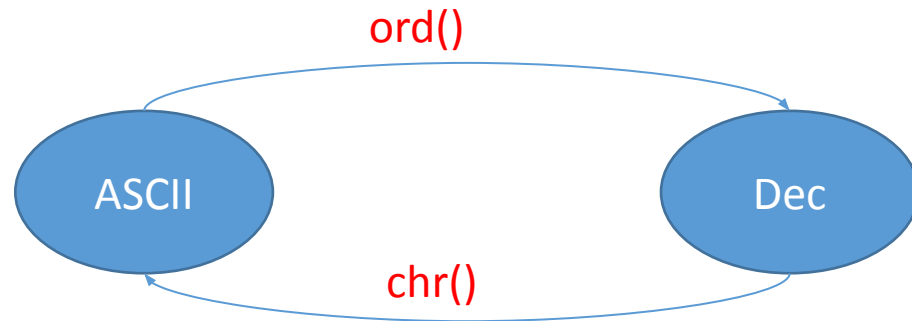
. is added to the end of the string

H.
Hello.

Conversão Decimal e ASCII **char** (Python version)

Decimal to an ASCII **char**

```
>>> ord("H")  
72  
>>> chr(72)  
'H'
```



Conversão de Número e ASCII (Python version)



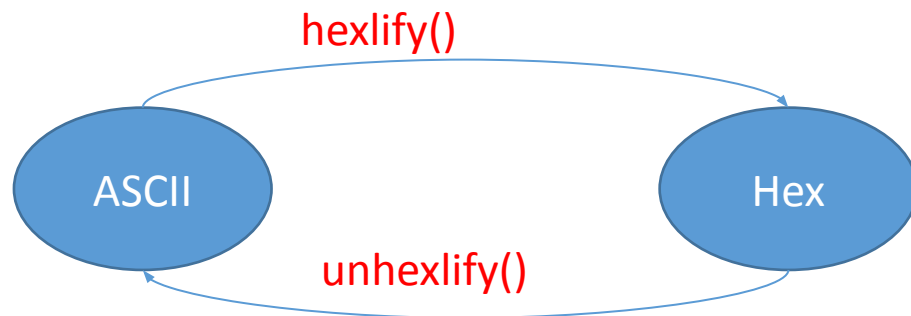
Python 3

ASCII to Hex

```
>>> import binascii
>>> binascii.hexlify(b'H')
b'48' ←
>>> binascii.hexlify(b'Hello')
b'48656c6c6f' ←
```

Hex to ASCII

```
>>> binascii.unhexlify('48')
b'H' ←
>>> binascii.unhexlify('48656c6c6f')
b'Hello' ←
```



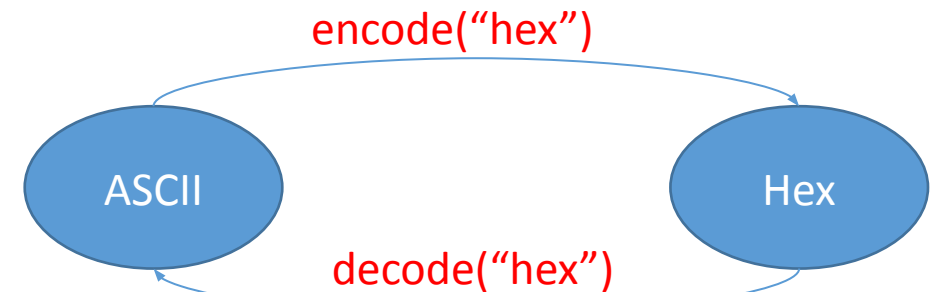
Python 2

ASCII to Hex

```
>>> "H".encode("hex")
'48'
>>> "Hello".encode("hex")
'48656c6c6f'
```

Hex to ASCII string

```
>>> "48".decode("hex")
'H'
>>> "48656c6c6f".decode("hex")
'Hello'
```



Qual é a diferença entre ASCII e UTF-8 Unicode?

- Both are character encoding standards used for representing characters
 - letters, numbers, symbols, etc.
 - use a series of bits (1s and 0s).
- Length difference
 - ASCII uses 7 bits (128 characters)
 - UTF-8 uses variable-length encoding (1,112,064 characters)
- UTF-8 is a superset of ASCII
 - all ASCII characters have the same representation in UTF-8.
 - UTF-8 can represent a wider range of characters, including non-English characters

Unicode

Unicode, formally The Unicode Standard is an information technology **standard** for the consistent **encoding, representation**, and handling of text expressed in most of the **world's** writing systems.

<https://en.wikipedia.org/wiki/Unicode>



UNICODE
Logo of the Unicode Consortium

Alias(es)	Universal Coded Character Set (UCS, ISO/IEC 10646)
Language(s)	International
Standard	Unicode Standard
Encoding formats	UTF-8 UTF-16 GB18030 Less common: UTF-32 BOCU SCSU Obsolete: UTF-7
Preceded by	ISO/IEC 8859, various others

Official website 
Official, technical website 
25

V · T · E

Conversão de data

Unix time

- Unix time, or POSIX time, is a system for describing points in time.
- It is the number of seconds that have elapsed since the Unix epoch,
- The Unix epoch is 00:00:00 [UTC](#) on 1 January 1970

Current time to Unix time

```
(kali㉿kali)-[~]  
$ date  
Mon Jan 31 10:38:09 AM EST 2022  
  
(kali㉿kali)-[~]  
$ date +%S  
1643643491
```

Unix time to readable time

```
(kali㉿kali)-[~]  
$ date -d @1643643491  
Mon Jan 31 10:38:11 AM EST 2022  
  
(kali㉿kali)-[~]  
$ date -d @1643643491 --utc  
Mon Jan 31 03:38:11 PM UTC 2022
```

Unix time (Python version)

0 second => UTC Jan, 1st, 1970

```
>>> from datetime import datetime
>>> datetime.utcfromtimestamp(0)
datetime.datetime(1970, 1, 1, 0, 0)
```

- A **Python module** is a .py file that contains Python code.
- A **Python module** contains multiple functions
- Use keyword from and import if you only want to import a few functions,

```
from [module] import [function1, function2]
```

UTC Jan, 1st, 1970 => 0 second

```
>>> from datetime import datetime, timezone
>>> d=datetime(1970, 1, 1, 0, 0, tzinfo=timezone.utc)
>>> d.timestamp()
0.0 ←
>>>
```

datetime(year, month, day, hour, minute, second, microsecond)

format

Get current time (datetime Module)

```
>>> from datetime import datetime, timezone
>>> datetime.now()
datetime.datetime(2023, 1, 29, 11, 3, 14, 480338)
>>> datetime.now(timezone.utc)
datetime.datetime(2023, 1, 29, 16, 3, 17, 190388, tzinfo=datetime.timezone.utc)
>>> sec = datetime.now().timestamp()
>>> print(sec)
1675009654.429736
```

5 hours difference (EST vs UTC)

Get the current time (time Module)

```
>>> import time
>>> time.ctime()
'Sun Jan 29 11:13:47 2023'
>>> time.time()
1675008832.9003756
```

Get the current time in a human-readable format

Get the current time in seconds. It returns the number of seconds since Jan 1, 1970, 00:00:00.

Covert seconds to a readable time (datetime Module)

```
>>> from datetime import datetime
>>> datetime.fromtimestamp(1675008832)
datetime.datetime(2023, 1, 29, 11, 13, 52)
>>>
```

our default time zone is EST. Pay attention to the **5** hours difference

If the timestamps are in UTC timezone (a common way to store dates) you should use

```
>>> datetime.utcfromtimestamp(1675008832)
datetime.datetime(2023, 1, 29, 16, 13, 52)
```

Mac "epoch time" - Cocoa Core Date

- Mac uses a different "epoch time" than the UNIX epoch time.
 - Cocoa Core Date
- Mac OS epoch date is Midnight January 1, 2001, UTC
 - Cocoa Core Data time is the number of seconds elapsed since January 1, 2001, 00:00:00 UTC,
- Different is 31 years (978307200 seconds)

Write a function to compute the difference between Unix time and iOS time

```
(student@kalit01)-[~]  
$ leafpad time_diff.py
```

```
time_diff.py  
File Edit Search Options Help  
from datetime import datetime, timezone, timedelta  
  
date1 = datetime(1970, 1, 1, tzinfo=timezone.utc)  
date2 = datetime(2001, 1, 1, tzinfo=timezone.utc)  
  
diff = date2 - date1  
seconds = diff.total_seconds()  
print(f"The difference between Unix time and iOS Cocoa Core Data Time is {seconds}")
```

```
(student@kalit01)-[~]  
$ python3 time_diff.py  
The difference between Unix time and iOS Cocoa Core Data Time is 978307200.0
```


Código Hash

Comandos Linux

```
(student@kalit01)-[~/WEP40]
$ echo -n hello | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -

(student@kalit01)-[~/WEP40]
$ echo -n hello | md5sum
5d41402abc4b2a76b9719d911017c592 -
```

- echo appends a `\n`
- Use the `-n` argument to omit the trailing linebreak

Python

```
1 import hashlib
2 hashlib.sha1("hello".encode('utf-8')).hexdigest()
3
```

✓ 0.0s

```
'aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d'
```

```
1 hashlib.md5("hello".encode('utf-8')).hexdigest()
```

✓ 0.0s

```
'5d41402abc4b2a76b9719d911017c592'
```

Dados armazenados na memória

Endianness

- Endianness is a term that describe the order in which a sequence of bytes (byte order) is stored in memory.
- Little-endian
 - is an order in which the "little end" (least significant value in the sequence) is stored first.
- Big-endian
 - is an order in which the "big end" (most significant value in the sequence) is stored first (at the lowest storage address).

Little Endian

Address 0x100 0x101 0x102 0x103

0x04	0x03	0x02	0x01
------	------	------	------

Data

0x01020304

Least significant value

Big Endian

Address 0x100 0x101 0x102 0x103

0x01	0x02	0x03	0x04
------	------	------	------

Check Byte order

```
(kali㉿kali)-[~]  
$ lscpu  
Architecture:          x86_64  
CPU op-mode(s):        32-bit, 64-bit  
Address sizes:          48 bits physical, 48 bits virtual  
Byte Order:             Little Endian  
CPU(s):                 6  
On-line CPU(s) list:    0-5  
Vendor ID:              AuthenticAMD  
Model name:             AMD Ryzen Threadripper 3970X 32-Core Processor  
CPU family:             23  
Model:                  49  
Thread(s) per core:     1  
Core(s) per socket:     6  
Socket(s):              1  
Stepping:               0  
BogoMIPS:               7800.00
```